Forensic Analysis & Remediation of Web-Based Attack Vectors: Final Defensive Case Study

1. Executive Summary

This project is my final blue-team rewrite of the Cisco Ethical Hacker Final Capstone, which I completed in a controlled lab environment. The original capstone focused on solving the challenges from an attacker point of view, but in this version not only did I execute the tasks, I analyzed and evaluated them from a defender's perspective.

Across the capstone, I identified four main security issues. The most serious was a SQL injection issue that exposed user data, revealed a password hash, and eventually led to successful SSH access using cracked credentials. The other findings involved exposed web directories, anonymous Samba share access, and useful internal path information being revealed through packet capture analysis.

What stood out to me most in this lab was that none of the weaknesses were extremely complex on their own. The real problem was that they reduced attacker effort, exposed sensitive information, or removed basic security boundaries. From a blue-team perspective, that is exactly why these issues matter.

---

2. Lab Scope and Environment

- Lab type - Controlled educational lab
- Capstone - Cisco Ethical Hacker Final Capstone
- Attacker/analyst machine - Preconfigured Kali Linux VM
- Virtualization platform - Oracle VirtualBox
- Networks observed during the capstone - 10.6.6.0/24 and 172.17.0.0/24
- Goal of this report - Reframe the completed offensive capstone as a defender-focused case study

All activities in this project were carried out inside an authorized training environment. No real-world systems or networks were involved or targeted in the process.

---

3. Method

I first completed the capstone normally and collected screenshots during reconnaissance, exploitation, and phase completion. After that, I reviewed each phase again from a blue-team perspective and asked the following questions:

- What security weakness made this possible?
- What was the actual attack path from start to finish?
- What defensive control failed?
- How serious would this be from a defender point of view?
- What should be monitored?
- What should be fixed first?
- How would I check that the fix actually worked?

This report is based on that review.

---

4. Severity Approach

I did not use full formal enterprise risk scoring for this project. Instead, I used a simpler model based on:

- Exploitability - how easy it was to abuse
- Impact - what the attacker gained
- Exposure - how much of a security boundary was affected
- Follow-on potential - whether it could lead to more access or more damage

Severity levels used in this report:

- Critical
- High
- Medium
- Low

Priority levels used in this report:

- P1 - fix first
- P2 - fix soon after
- P3 - important, but lower urgency than the others

---

5. Findings Overview

Finding 1 - Improper Input Handling Allowed SQL Injection, Credential Exposure, and Follow-On SSH Access

- Severity - Critical
- Priority - P1

Finding 2 - Directory Listing and Exposed Web Content Allowed Unauthorized File Discovery

- Severity - High
- Priority - P2

Finding 3 - Anonymous Samba Share Access Allowed Unauthorized File Retrieval

- Severity - High
- Priority - P2

Finding 4 - Packet Capture Analysis Revealed Internal Resource Information and Reduced Attacker Effort

- Severity - Medium
- Priority - P3

---

6. Detailed Findings

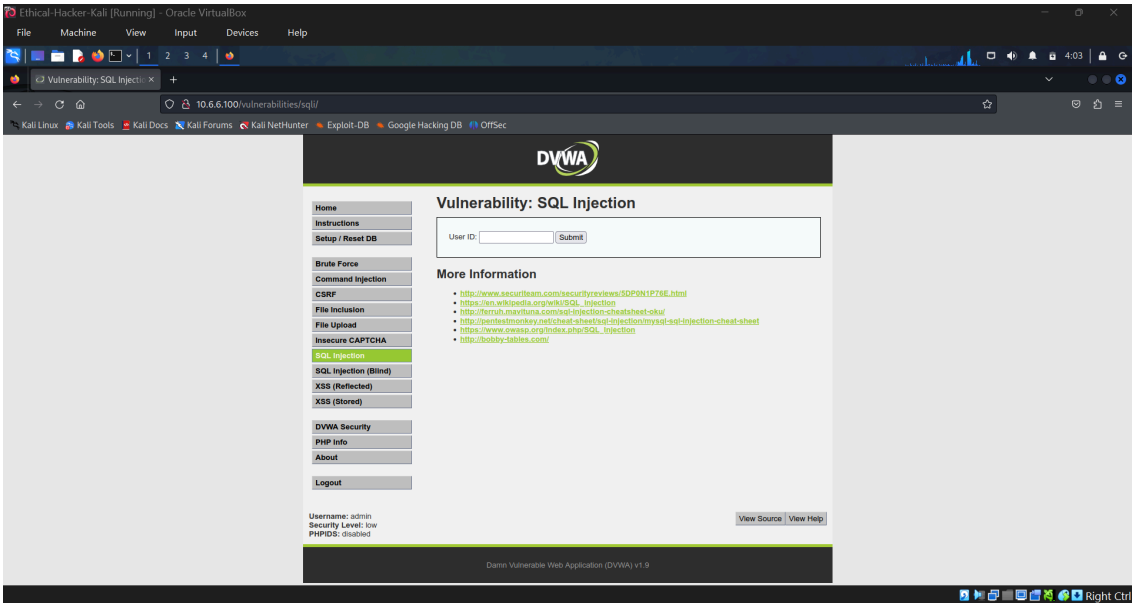
Finding 1 - Improper Input Handling Allowed SQL Injection, Credential Exposure, and Follow-On SSH Access

Summary

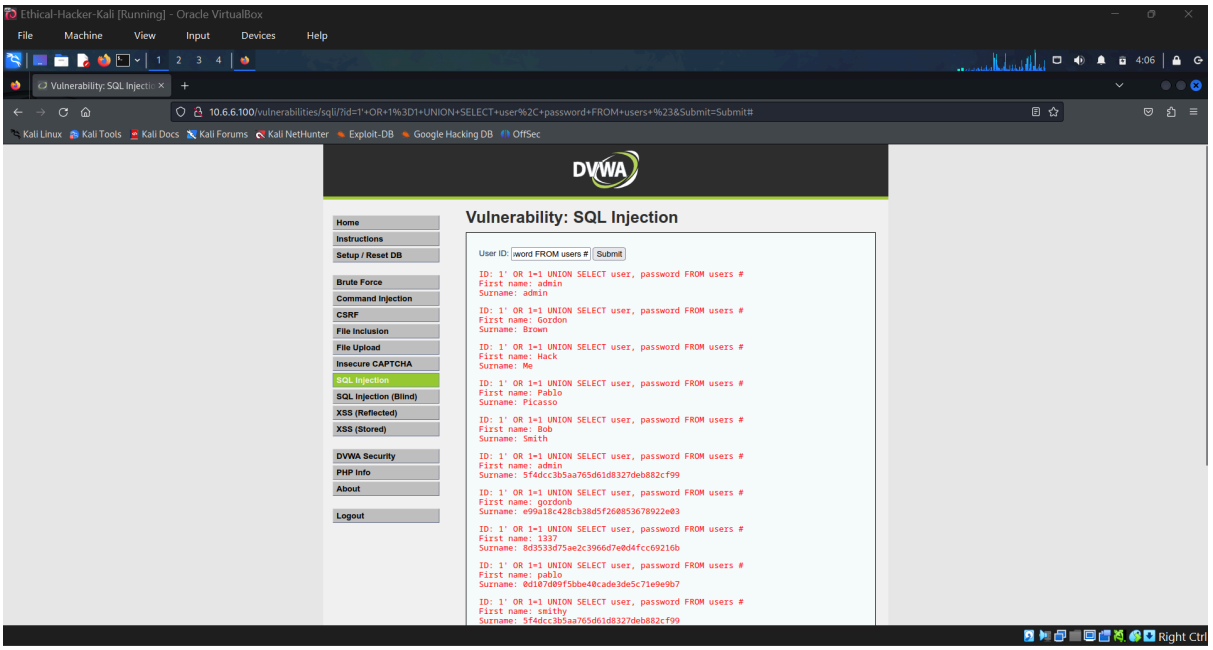
The first phase started with a vulnerable web input field. By using SQL injection, I was able to make the application return database information that should not have been exposed. That included the username '*gordonb*' and a password hash. After cracking the hash, I used the recovered password to log in over SSH and retrieve the simulated sensitive data.

This was the strongest attack chain in the whole capstone because it moved from a web application issue into actual system access.

Evidence



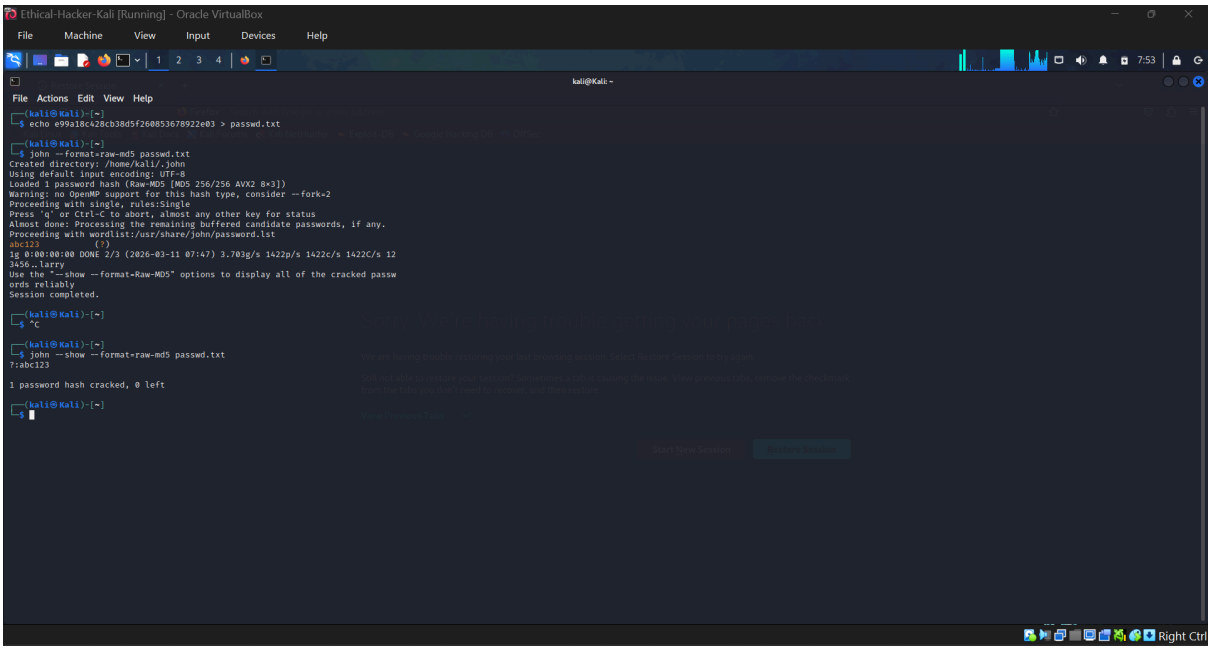
(vulnerable SQL input field identified)



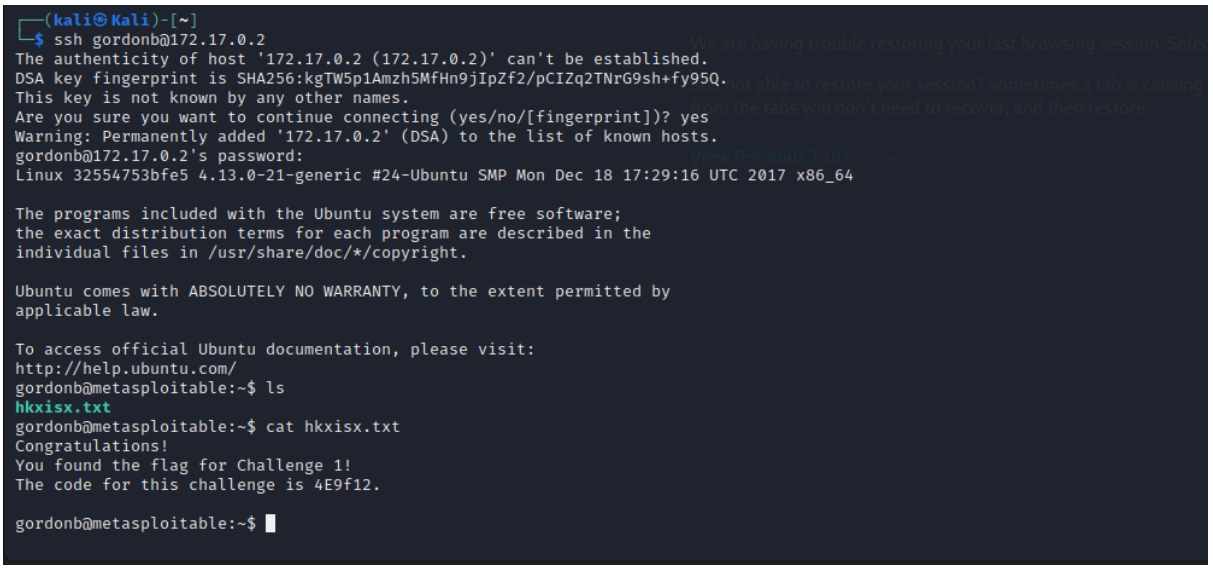
(user data returned through SQL injection)

```
ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: Gordon
Surname: Brown

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: Hack
Surname: Me

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: Pablo
Surname: Picasso

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: Bob
Surname: Smith

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03
```

(username and hash exposed)



(password hash cracked)



(SSH login and Challenge 1 flag)

Attack Path

- Vulnerable input field
- SQL injection
- Database data returned in the application response
- Username and password hash exposed
- Hash cracked offline
- SSH login using recovered credentials
- Challenge 1 flag retrieved

Root Cause

The main issue here was insecure handling of user input. The application allowed input to affect the SQL query instead of safely treating it as data. A second issue was that once credential-related data was exposed, it was useful enough to be cracked and reused.

Why It Matters to Defenders

From a blue-team perspective, this was not just “a web bug.” It became a full attack path:

- Web application abuse
- Data exposure
- Credential compromise
- Unauthorized remote access

That makes it much more serious than a simple input validation issue. If something similar happened in a real environment, it could lead to account compromise, lateral movement, or access to additional systems.

MITRE ATT&CK Mapping

This finding can be reasonably linked to:

- *T1190 - Exploit Public-Facing Application*
- *T1078 - Valid Accounts*

Severity Reasoning

- Exploitability - High
- Impact - High
- Exposure - Broad
- Follow-on potential - High

Severity - Critical
Priority - P1

I rated this as Critical because it directly led to credential exposure and successful host access.

Detection Ideas

If I were looking at this from a defensive monitoring point of view, I would want to check:

- Web server logs for suspicious input patterns.

- Repeated requests containing SQL-like characters or malformed input.
- Unusual responses where database content is returned.
- SSH login activity soon after suspicious web requests.
- Attempts to use the exposed account after the application issue is triggered.

To actively monitor for a recurrence of this attack vector, the following Sigma rule was developed to detect common SQL injection patterns within web server logs:

```
title: Detect SQL Injection Patterns in Apache Logs
status: experimental
description: Detects common SQL injection patterns (UNION, SELECT, information_schema) in web server
logsource:
    category: webserver
detection:
    selection:
        cs-uri-query|contains:
            - "UNION SELECT"
            - "ORDER BY"
            - "information_schema"
            - "1=1"
    condition: selection
falsepositives:
    - Authorized Penetration Testing / DAST Scanning
level: critical
tags:
    - attack.t1190
```

Basic Triage Logic

If suspicious SQL injection activity was seen, I would:

1. Identify the affected page or parameter.
2. Review the request pattern and source.
3. Check whether sensitive data was returned in the response.
4. Review authentication logs for related follow-on access.
5. Determine whether any credentials need to be rotated immediately.

Containment Ideas

- Temporarily disable or restrict the vulnerable input page.
- Block suspicious request patterns if possible.
- Rotate exposed credentials immediately.
- Review and restrict SSH access for affected accounts.

Remediation

- Use parameterized queries or prepared statements.
- Validate input safely on the server side.
- Avoid exposing database errors or sensitive output to users.
- Store passwords using strong modern hashing.

- Minimize the privileges of the database account used by the web app.
- Review whether password-based SSH access should be reduced.

Validation Checks

After fixing the issue, I would verify that:

- The old SQL injection input no longer works.
- The application no longer returns database data.
- The exposed credentials are no longer valid.
- Suspicious input is safely handled or rejected.
- SSH access with the old recovered password fails.

Finding 2 - Directory Listing and Exposed Web Content Allowed Unauthorized File Discovery

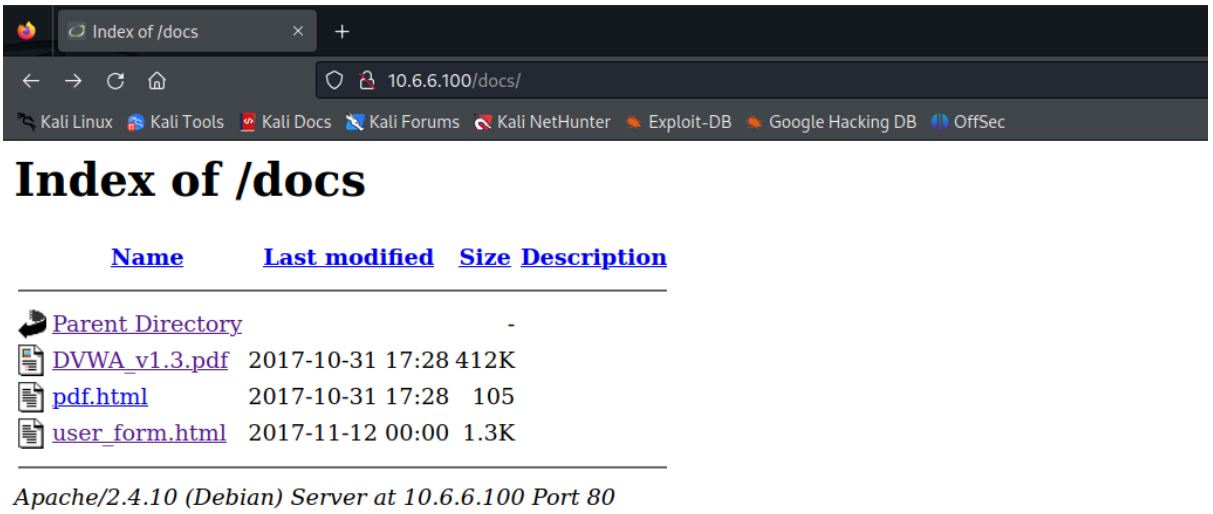
Summary

In the second challenge, web reconnaissance revealed that directory indexing was enabled. I was able to browse the '*/docs/* 'directory directly in the browser and continue navigating through exposed content until I located the Challenge 2 file and flag.

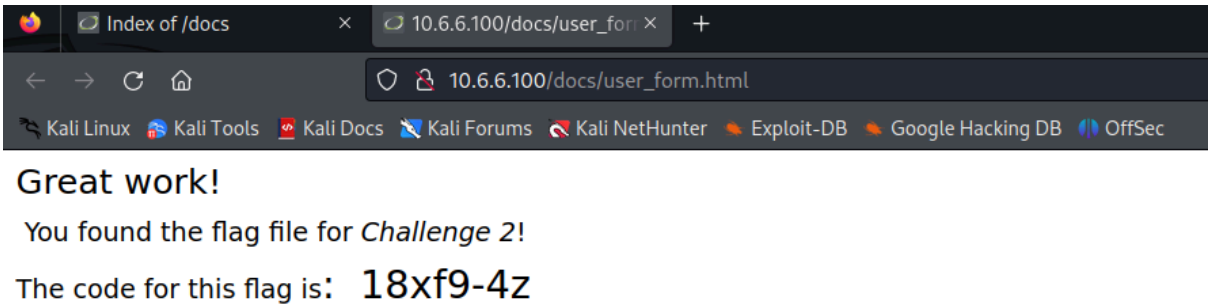
This was not a complicated exploit, which is exactly why it is important. The exposure came from weak web hardening rather than advanced attacker capability.

Evidence

(directory indexing identified)



(indexed directory visible in browser)



(Challenge 2 flag visible through exposed content)

Attack Path

- Web reconnaissance
- Directory indexing identified
- Indexed directory opened in browser
- Files and paths browsed directly
- Challenge file located

- Challenge 2 flag found

Root Cause

The main problem was web server misconfiguration. Directory listing was enabled, and sensitive or unnecessary content was placed in a location that could be reached directly through the web server.

Why It Matters to Defenders

This type of issue gives attackers a lot of visibility with very little effort. Once directory indexing is available, an attacker can quickly learn:

- File names
- Folder structure
- Documentation paths
- Possible application components
- Useful content for later attacks

Even if no exploit code is used, it still creates real security exposure.

MITRE ATT&CK Mapping

This finding can be linked to:

- *T1595 - Active Scanning*
- *T1083 - File and Directory Discovery*

Severity Reasoning

- Exploitability - **High**
- Impact - **Medium**
- Exposure - **Moderate**
- Follow-on potential - **Medium**

Severity - High
Priority - P2

I rated this as High because it allowed direct discovery of sensitive content with very low attacker effort.

Detection Ideas

A defender could look for:

- Repeated browsing of indexed directories.

- Requests to unusual paths such as '*/docs/*'.
- Fast sequential access to multiple files in the same folder.
- Enumeration-like behavior against non-public directories.

Basic Triage Logic

If this type of activity appeared in logs, I would:

1. Identify the directory being accessed.
2. Confirm whether it should ever be public.
3. Review which files were viewed or downloaded.
4. Check whether related paths were also probed.
5. Assess whether the exposure created follow-on risk.

Containment Ideas

- Disable directory indexing immediately.
- Temporarily block access to the exposed path.
- Move sensitive content out of web-accessible folders.

- Review adjacent directories for similar exposure.

Remediation

- Disable directory listing on the web server.
- Keep sensitive files outside the web root.
- Restrict access to directories that are not meant to be public.
- Regularly review web-exposed content.
- Harden default web server settings.

Validation Checks

After remediation, I would verify that:

- The '*/docs/*' path no longer shows an index.
- Sensitive files are no longer directly accessible.
- Only intended public files remain reachable.
- The same browsing path no longer leads to the challenge content.

<u>Finding 3 - Anonymous Samba Share Access Allowed Unauthorized File Retrieval</u>

Summary

In the third challenge, network scanning identified SMB-related services running on the target. After enumerating the available shares, I found that anonymous access was allowed. That meant I could browse the share contents and retrieve a file without valid credentials. From there, I was able to obtain the Challenge 3 flag.

This finding is a strong example of how weak file-sharing configuration can create immediate access problems even without exploitation in the traditional sense.

Evidence

```
┌──(kali㉿Kali)-[~]
└─$ nmap 10.6.6.0/24
Starting Nmap 7.94 ( https://nmap.org ) at 2026-03-11 08:16 UTC
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 10.6.6.1
Host is up (0.00091s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT   STATE SERVICE
22/tcp open  ssh

Nmap scan report for webgoat.vm (10.6.6.11)
Host is up (0.00095s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT     STATE SERVICE
8080/tcp open  http-proxy
8888/tcp open  sun-answerbook
9001/tcp open  tor-orport

Nmap scan report for juice-shop.vm (10.6.6.12)
Host is up (0.00098s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT     STATE SERVICE
3000/tcp open  ppp

Nmap scan report for dvwa.vm (10.6.6.13)
Host is up (0.00097s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT   STATE SERVICE
80/tcp open  http

Nmap scan report for mutillidae.vm (10.6.6.14)
Host is up (0.00081s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT     STATE SERVICE
3306/tcp open  mysql

Nmap scan report for gravemind.vm (10.6.6.23)
Host is up (0.00070s latency).
Not shown: 994 closed tcp ports (conn-refused)
PORT    STATE SERVICE
21/tcp  open  ftp
22/tcp  open  ssh
53/tcp  open  domain
80/tcp  open  http
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds

Nmap scan report for 10.6.6.100
Host is up (0.00083s latency).
```

(SMB services identified)

```
Nmap scan report for 10.6.6.100
Host is up (0.00083s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT   STATE SERVICE
80/tcp open  http

Nmap done: 256 IP addresses (7 hosts up) scanned in 3.09 seconds

┌──(kali㉿Kali)-[~]
└─$
```

(supporting SMB exposure evidence)

```
┌──(kali㉿Kali)-[~]
└─$ nmap -sV  --script=smb-enum-shares 10.6.6.23
Starting Nmap 7.94 ( https://nmap.org ) at 2026-03-11 08:22 UTC
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for gravemind.vm (10.6.6.23)
Host is up (0.00019s latency).
Not shown: 994 closed tcp ports (conn-refused)
PORT    STATE SERVICE     VERSION
21/tcp  open  ftp         vsftpd 3.0.3
22/tcp  open  ssh         OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
53/tcp  open  domain      ISC BIND 9.11.5-P4-5.1+deb10u5 (Debian Linux)
80/tcp  open  http        nginx 1.14.2
|_http-server-header: nginx/1.14.2
139/tcp open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
Service Info: Host: GRAVEMIND; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
| smb-enum-shares:
|   account_used: <blank>
|   \\10.6.6.23\IPC$:
|     Type: STYPE_IPC_HIDDEN
|     Comment: IPC Service (Samba 4.9.5-Debian)
|     Users: 1
|     Max Users: <unlimited>
|     Path: C:\tmp
|     Anonymous access: READ/WRITE
|   \\10.6.6.23\print$:
|     Type: STYPE_DISKTREE
|     Comment: Printer Drivers
|     Users: 0
|     Max Users: <unlimited>
|     Path: C:\var\lib\samba\printers
|     Anonymous access: READ/WRITE
|   \\10.6.6.23\workfiles:
|     Type: STYPE_DISKTREE
|     Comment: Confidential Workfiles
|     Users: 0
|     Max Users: <unlimited>
|     Path: C:\var\spool\samba
|_    Anonymous access: READ/WRITE

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 19.92 seconds

┌──(kali㉿Kali)-[~]
└─$
```

(anonymous access to share confirmed)

```
┌──(kali㉿Kali)-[~]
└─$ smbclient //10.6.6.23/print$
Password for [WORKGROUP\kali]:
Anonymous login successful
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Mon Aug 14 09:40:01 2023
  ..                                  D        0  Mon Aug 30 05:00:05 2021
  IA64                                D        0  Mon Sep  2 13:39:42 2019
  x64                                 D        0  Mon Aug 30 05:00:05 2021
  W32X86                              D        0  Mon Aug 30 05:00:05 2021
  W32MIPS                             D        0  Mon Sep  2 13:39:42 2019
  W32ALPHA                            D        0  Mon Sep  2 13:39:42 2019
  COLOR                               D        0  Mon Sep  2 13:39:42 2019
  W32PPC                              D        0  Mon Sep  2 13:39:42 2019
  WIN40                               D        0  Mon Sep  2 13:39:42 2019
  OTHER                               D        0  Tue Aug 10 00:00:00 2021
  color                               D        0  Mon Aug 30 05:00:05 2021

                38497656 blocks of size 1024. 9078692 blocks available
smb: \> cd OTHER
smb: \OTHER\> get taxes.txt
getting file \OTHER\taxes.txt of size 103 as taxes.txt (1.7 KiloBytes/sec) (average 1.7 KiloBytes/sec)
smb: \OTHER\> exit

┌──(kali㉿Kali)-[~]
└─$ cat taxes.txt
Congratulations!
You found the flag for Challenge 3!
The code for this challenge is A9!15wa2.


┌──(kali㉿Kali)-[~]
└─$
```

(file retrieval and challenge 3 flag)

Attack Path

- Network scan
- SMB services discovered
- Shares enumerated
- Anonymous access confirmed
- File listing and retrieval
- Challenge 3 flag obtained

Root Cause

The main issue was overly permissive Samba configuration. Anonymous or guest access was allowed, which removed the authentication barrier that should have protected the share.

Why It Matters to Defenders

File shares often contain useful internal data. If anonymous access is possible, an attacker does not need to steal credentials first. They can go straight to enumeration and data access.

From a defender perspective, this kind of weakness can lead to:

- Unauthorized file access.
- Internal data exposure.
- Easier reconnaissance.
- Possible staging for later activity.

MITRE ATT&CK Mapping

This finding can be linked to:

- *T1135 - Network Share Discovery*
- *T1039 - Data from Network Shared Drive*

Severity Reasoning

- Exploitability - High
- Impact - High
- Exposure - Moderate
- Follow-on potential - Medium to High

Severity - High
Priority - P2

I rated this as High because authentication was effectively bypassed through configuration, and files could be retrieved directly.

Detection Ideas

Useful things to monitor would include:

- Anonymous or guest SMB sessions.
- Repeated share enumeration.
- Unusual listing of multiple shares from one host.
- File access from sessions that should not exist without authentication.

Basic Triage Logic

If suspicious SMB activity was detected, I would:

1. Identify the source host.
2. Identify which share was accessed.
3. Determine whether access was anonymous or authenticated.
4. Check whether files were only listed or actually retrieved.
5. Look for activity against other shares from the same source.

Containment Ideas

- Disable guest or anonymous access immediately.
- Restrict network access to the SMB service if needed.
- Temporarily remove or hide exposed shares.
- Monitor for repeated connection attempts after access is removed.

Remediation

- Require authentication for all shares.
- Apply least-privilege permissions.
- Remove unnecessary shares.
- Review share access regularly.
- Restrict SMB exposure where possible.
- Enable useful logging on the file-sharing service.

Validation Checks

After remediation, I would verify that:

- Anonymous '*smbclient*' access fails.
- Unauthenticated users cannot list share contents.
- Only approved users can access the share.
- The same retrieval path no longer works.

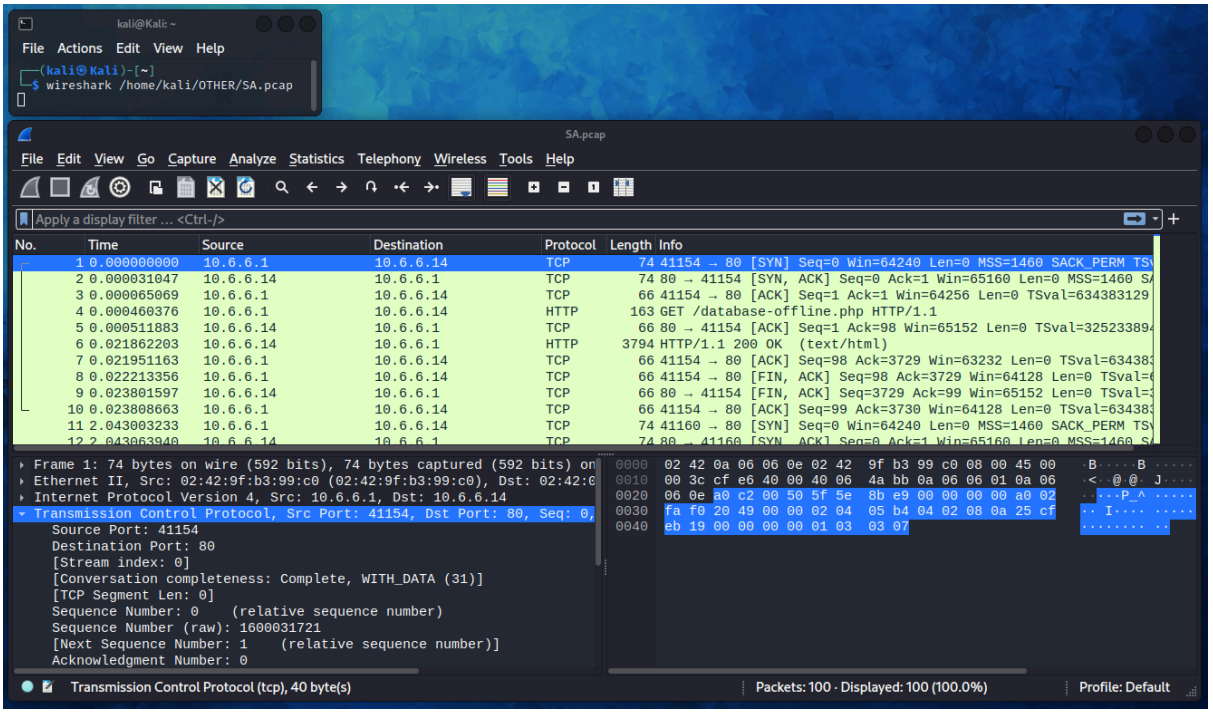
Finding 4 - Packet Capture Analysis Revealed Internal Resource Information and Reduced Attacker Effort

Summary

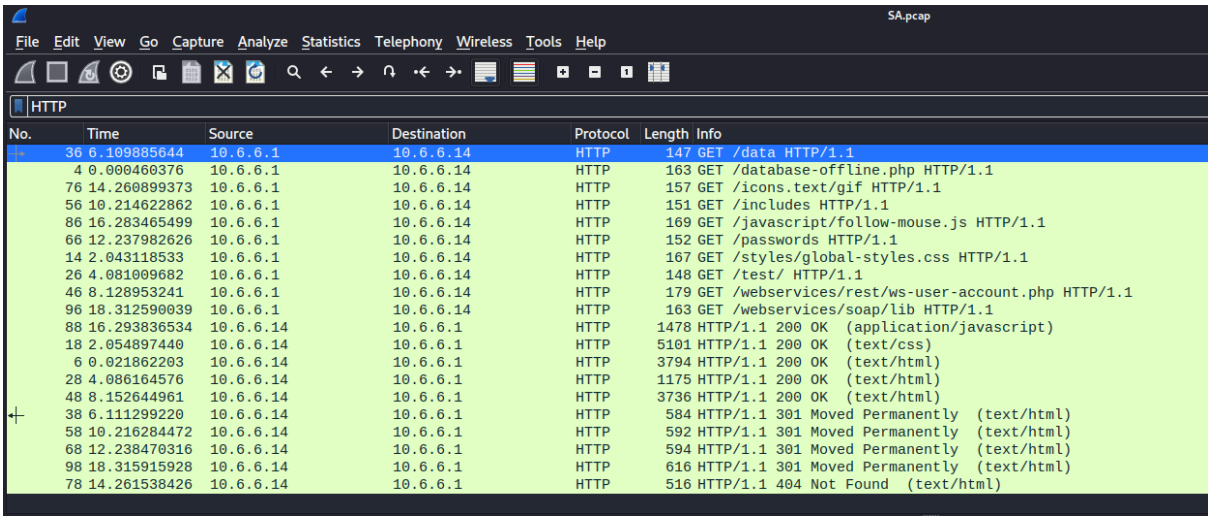
In the fourth challenge, I opened the '*SA.pcap*' file in Wireshark and reviewed the traffic. Looking at the HTTP traffic revealed a host and path that were relevant: 10.6.6.14 and /data. That pointed to a critical internal location. The PCAP data revealed an anomalous HTTP GET request targeting *http://10.6.6.14/data/accounts.xml*. By correlating the packet sequence numbers and destination IP, I successfully mapped the adversary's exact reconnaissance path to this internal resource."

This finding was different from the others because it was more about information disclosure and reconnaissance value than direct exploitation.

Evidence



(packet capture opened in Wireshark)



(internal host and path identified)

The code for Challenge 4 at http://10.6.6.14/data/accounts.xml. The flag value is in the Signature field for the employee ID="0" record.

(final answer path confirmed)

Attack Path

- Packet capture opened.
- HTTP traffic reviewed.
- Relevant internal host and path identified.
- Resource location inferred.
- Exact location confirmed.

Root Cause

The main issue was that useful internal information was visible in traffic and preserved in the packet capture. That meant the PCAP itself became a source of intelligence.

Why It Matters to Defenders

This finding shows that technical artifacts can become security-sensitive too. Even when a packet capture does not directly hand over credentials, it can still expose:

- Internal IP addresses
- File locations
- Application paths
- Useful clues for later targeting

That reduces attacker effort and helps them focus on the right place faster.

MITRE ATT&CK Mapping

This finding can be linked to:

- *T1040 - Network Sniffing*
- *T1592 - Gather Victim Host Information*

Severity Reasoning

- Exploitability - Medium
- Impact - Medium
- Exposure - Limited to Moderate
- Follow-on potential - Medium

Severity - Medium
Priority - P3

I rated this as Medium because it did not directly give deep access by itself, but it clearly made reconnaissance easier.

Detection Ideas

A defender could look for:

- unusual requests to sensitive internal paths,
- repeated access to internal files once the path becomes known,
- continued use of cleartext HTTP where it exposes useful operational information,
- packet captures or share artifacts that contain sensitive internal details.

Basic Triage Logic

If an exposed internal path was identified, I would:

1. confirm what host and file/path were revealed,
2. check whether the resource is still reachable,
3. review whether it was accessed afterward,
4. determine whether the issue comes from traffic exposure, application design, or both,
5. decide whether the path or file should be restricted or moved.

Containment Ideas

- Restrict access to the exposed path.
- Move or protect sensitive internal files.

- Reduce sharing of raw packet captures containing sensitive information.
- Review whether the service should still expose that path at all.

Remediation

- Reduce unnecessary internal path disclosure.
- Avoid exposing sensitive file locations through client-visible traffic.
- Protect packet captures that contain operational details.
- Use more secure transport where appropriate.
- Review which internal resources are externally or broadly reachable.

Validation Checks

After remediation, I would verify that:

- The exposed path is no longer openly accessible.
- Sensitive internal file locations are no longer visible in normal traffic.
- Packet captures shared for learning or review are sanitized if needed.
- Requests to the sensitive resource can be properly logged and reviewed.

---

7. Risk Prioritization

- Priority 1 - Finding 1

  This should be fixed first because it created the clearest compromise path from web input to host access.

- Priority 2 - Findings 2 and 3

  These should come next because both expose sensitive content with low attacker effort. One affects the web layer and one affects file-sharing access, but both remove important defensive barriers.

- Priority 3 - Finding 4

  This is still important, but compared with the others it is more of a reconnaissance and information-disclosure issue than a direct compromise path.

Recommended Fix Order

1. Fix the SQL injection issue and rotate exposed credentials.
2. Disable directory indexing and remove exposed web content.
3. Disable anonymous Samba access and tighten share permissions.

4. Reduce internal path exposure and treat packet captures as sensitive artifacts.

---

8. Detection and Monitoring Summary

Finding 1

- Main logs to review - web logs, application logs, SSH authentication logs
- Main signs - suspicious input, abnormal responses, related SSH access

Finding 2

- Main logs to review - web access logs
- Main signs - indexed directory access, browsing of unusual file paths

Finding 3

- Main logs to review - Samba logs, host authentication logs, file access logs
- Main signs - anonymous sessions, share enumeration, suspicious file retrieval

Finding 4

- Main logs to review - web logs, proxy logs, network review artifacts
- Main signs - access to exposed internal paths, continued cleartext disclosure of useful resource details

---

9. Main Defensive Lessons Learned

This capstone reinforced a few important blue-team lessons for me.

1.) Small weaknesses can become bigger problems when they connect to each other. The SQL injection issue was the best example of that.

2.) Misconfiguration can be just as dangerous as a more “technical” vulnerability. The directory listing and Samba findings were both largely configuration problems.

3.) Information exposure matters. Even when a weakness does not give instant access, it can still reduce attacker effort and make later attacks easier.

Finally, defensive thinking is not only about detecting malicious actions after they happen. It is also about reducing exposure, enforcing access control, and making sure basic hardening is in place before an attacker gets the chance.

---

10. Limitations

This project was done in a lab, not a real enterprise environment. Because of that:

- I did not have a real SIEM deployment
- I did not have enterprise EDR or centralized monitoring
- the detection ideas in this report are proposed defensive ideas, not proof of a live SOC implementation
- the containment and validation sections are based on how I would handle the findings in a practical blue-team workflow, not on a production change process

I think keeping that boundary clear makes the project stronger and more honest.

---

11. Conclusion

This final version of the project shows how a completed offensive capstone can be turned into a much stronger blue-team case study. Instead of just showing how each challenge was solved, this report explains what security weaknesses made it possible, why it matters from a defender point of view, what should be fixed first, and how the fixes could be checked afterward.

The most serious issue was the SQL injection chain because it led to credential exposure and successful SSH access. The other findings showed how weak web hardening, weak file-sharing permissions, and exposed internal traffic details can still create real defensive risk even when the attack itself is relatively simple.

Overall, this project helped me move from just completing a security lab to thinking more like a defender analyzing how and why the environment failed.

---

12. Disclaimer

This project was completed in an authorized educational lab environment for learning and portfolio purposes only. No unauthorized activity was performed against real systems or networks.